

React Component Life Cycle

Hands-on Lab Report

Objectives

- Explain the need and benefits of component life cycle
- Identify various life cycle hook methods
- List the sequence of steps in rendering a component

Explanation and Answers to Objectives

1. Need and Benefits of Component Life Cycle

Every React component moves through three phases during its existence: it is created and inserted into the DOM (mounting), it is updated in response to changes in props or state (updating), and it is eventually removed from the DOM (unmounting). The component life cycle refers to the series of methods React automatically invokes at each of these phases.

The need for the life cycle arises because a component often has to do more than just render markup — it may need to fetch data from a server, set up subscriptions or timers, measure the DOM, or clean up resources before it disappears. Life cycle methods give the developer a precise, predictable point in time to run such code, instead of guessing when the DOM is ready or when props have changed.

Benefits of the component life cycle include:

- Controlled data fetching – API calls can be made right after the component mounts, ensuring the DOM exists before data is used.
- Resource cleanup – timers, subscriptions, or event listeners can be removed before the component unmounts, preventing memory leaks.
- Performance optimization – methods like `shouldComponentUpdate()` let a component skip unnecessary re-renders.
- Error handling – `componentDidCatch()` lets a component catch errors in its child tree and show a fallback UI instead of crashing the whole app.
- Synchronization with external systems – life cycle hooks make it possible to keep the component in sync with things outside React's control, such as the browser DOM or third-party libraries.

2. Various Life Cycle Hook Methods

React's class component life cycle methods are grouped into four phases:

Mounting (component is being created and inserted into the DOM):

- `constructor(props)` – initializes state and binds methods.
- `static getDerivedStateFromProps(props, state)` – updates state based on changes in props before rendering.
- `render()` – returns the JSX to be displayed.
- `componentDidMount()` – runs right after the component is inserted into the DOM; ideal for data fetching, subscriptions, and DOM measurements.

Updating (component is being re-rendered due to changes in props or state):

- `static getDerivedStateFromProps(props, state)` – called again before every render.

- `shouldComponentUpdate(nextProps, nextState)` – decides whether the component should re-render.
- `render()` – re-renders the updated JSX.
- `getSnapshotBeforeUpdate(prevProps, prevState)` – captures information (e.g., scroll position) from the DOM before it is updated.
- `componentDidUpdate(prevProps, prevState, snapshot)` – runs after the update is committed to the DOM; used for reacting to prop/state changes.

Unmounting (component is being removed from the DOM):

- `componentWillUnmount()` – runs right before the component is destroyed; used to clean up timers, subscriptions, and listeners.

Error Handling (an error occurred during rendering, in a life cycle method, or in a child's constructor):

- `static getDerivedStateFromError(error)` – updates state so the next render shows a fallback UI.
- `componentDidCatch(error, info)` – logs error information and can trigger side effects such as displaying an alert.

3. Sequence of Steps in Rendering a Component

When a component is rendered for the first time (initial mount), the methods are called in the following order:

`constructor()` → `static getDerivedStateFromProps()` → `render()` → `componentDidMount()`

When a component re-renders because its props or state change (update phase), the order is:

`static getDerivedStateFromProps()` → `shouldComponentUpdate()` → `render()` → `getSnapshotBeforeUpdate()` → `componentDidUpdate()`

Finally, when the component is removed from the page (unmount phase), only one method runs:

`componentWillUnmount()`

In this lab, the sequence in action is: the `Posts` component is constructed, rendered for the first time with an empty `posts` array, then `componentDidMount()` fires and calls `loadPosts()`, which fetches data and updates the state. This state update triggers a re-render, so `render()` runs again — this time with the fetched posts — and the updated list appears on the page. If an error occurs anywhere in this process, `componentDidCatch()` intercepts it and displays an alert instead of crashing the application.

Overview

In this hands-on lab, you will learn how to:

- Implement the `componentDidMount()` hook
- Implement the `componentDidCatch()` life cycle hook

Prerequisites

The following is required to complete this hands-on lab:

- Node.js
- NPM
- Visual Studio Code

Notes

Estimated time to complete this lab: 60 minutes.

Steps

1. Create a new React application using the create-react-app tool with the name "blogapp".

```
npx create-react-app blogapp
```

2. Open the application using VS Code.

```
cd blogapp  
code .
```

3. Create a new file named Post.js in the src folder with the following properties:

```
import React, { Component } from 'react';  
  
class Post extends Component {  
  render() {  
    return (  
      <div>  
        <h2>{this.props.title}</h2>  
        <p>{this.props.body}</p>  
      </div>  
    );  
  }  
}  
  
export default Post;
```

Figure 1: Post class

4. Create a new class-based component named Posts inside Posts.js file.

```
import React, { Component } from 'react';  
import Post from './Post';  
  
class Posts extends Component {  
  // component logic added in subsequent steps  
}  
  
export default Posts;
```

Figure 2: Posts component

5. Initialize the component with a list of posts in the state of the component using the constructor.

```
constructor(props) {  
  super(props);  
  this.state = {  
    posts: []  
  };  
}
```

6. Create a new method in the component named loadPosts() which uses the Fetch API and assigns the result to the component state. Use the URL <https://jsonplaceholder.typicode.com/posts> to fetch the posts.

```
loadPosts() {  
  fetch('https://jsonplaceholder.typicode.com/posts')  
    .then(response => response.json())  
    .then(data => {  
      this.setState({ posts: data });  
    })  
    .catch(error => {
```

```

        throw new Error(error);
    });
}

```

Figure 3: loadPosts() method

7. Implement the componentDidMount() hook to call loadPosts(), which fetches the posts when the component mounts.

```

componentDidMount() {
    this.loadPosts();
}

```

Figure 4: componentDidMount() hook

8. Implement the render() method to display the title and body of each post on the HTML page using a heading and a paragraph respectively.

```

render() {
    return (
        <div>
            <h1>Blog Posts</h1>
            {this.state.posts.map(post => (
                <Post key={post.id} title={post.title} body={post.body} />
            ))}
        </div>
    );
}

```

Figure 5: render() method

9. Define a componentDidCatch() method to display any error occurring in the component as an alert message.

```

componentDidCatch(error, info) {
    alert(error);
}

```

Figure 6: componentDidCatch() hook

10. Add the Posts component to the App component.

```

import React from 'react';
import Posts from './Posts';

function App() {
    return (
        <div className="App">
            <Posts />
        </div>
    );
}

export default App;

```

11. Build and run the application using the npm start command.

```

npm start

```

Output

Running the application with "npm start" launches the blog app in the browser at localhost:3000. The `componentDidMount()` hook triggers `loadPosts()`, which fetches the list of posts from the JSONPlaceholder API and renders each post's title as a heading and its body as a paragraph, as shown in the screenshots below.

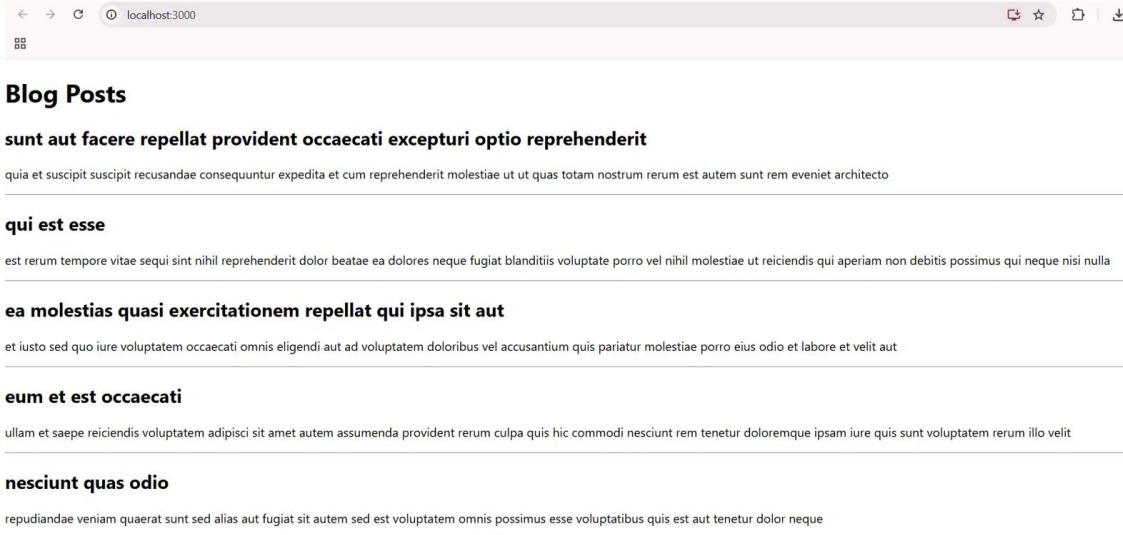


Figure 7: Blog Posts output — screenshot 1 of 7

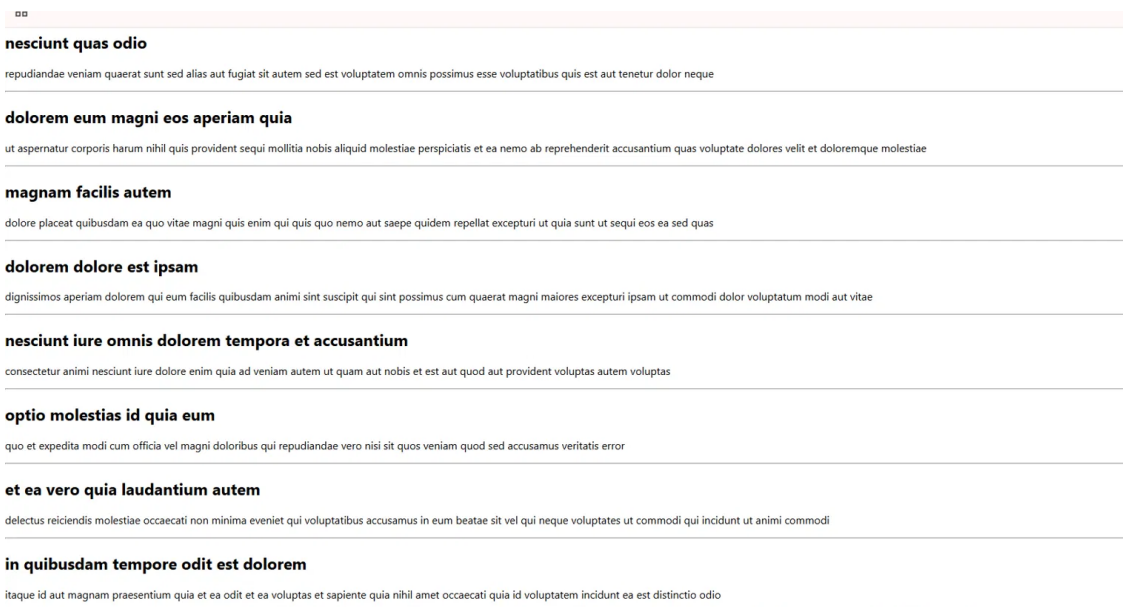


Figure 8: Blog Posts output — screenshot 2 of 7

magnam ut rerum iure

ea velit perferendis earum ut voluptatem voluptate itaque iusto totam pariatur in nemo voluptatem voluptatem autem magni tempora minima in est distinctio qui assumenda accusamus dignissimos officia nesciunt nobis

id nihil consequatur molestias animi provident

nisi error delectus possimus ut eligendi vitae placeat eos harum cupiditate facilis reprehenderit voluptatem beatae modi ducimus quo illum voluptas eligendi et nobis quia fugit

fuga nam accusamus voluptas reiciendis itaque

ad mollitia et omnis minus architecto odit voluptas doloremque maxime aut non ipsa qui alias veniam blanditiis culpa aut quia nihil cumque facere et occaecati qui aspernatur quia eaque ut aperiam inventore

provident vel ut sit ratione est

debitis et eaque non officia sed nesciunt pariatur vel voluptatem iste vero et ea numquam aut expedita ipsum nulla in voluptates omnis consequatur aut enim officiis in quam qui

explicabo et eos deleniti nostrum ab id repellendus

animi esse sit aut sit nesciunt assumenda eum voluptas quia voluptatibus provident quia necessitatibus ea rerum repudiandae quia voluptatem delectus fugit aut id quia ratione optio eos iusto veniam iure

eos dolorem iste accusantium est eaque quam

corporis rerum ducimus vel eum accusantium maxime aspernatur a porro possimus iste omnis est in deleniti asperiores fuga aut voluptas sapiente vel dolore minus voluptatem incidunt ex

enim quo cumque

ut voluptatum aliquid illo tenetur nemo sequi quo facilis ipsum rem optio mollitia quas voluptatem eum voluptas qui unde omnis voluptatem iure quasi maxime voluptas nam

Figure 11: Blog Posts output — screenshot 5 of 7

commodi ullam sint et excepturi error explicabo praesentium voluptas

odio fugit voluptatum ducimus earum autem est incidunt voluptatem odit reiciendis aliquam sunt sequi nulla dolorem non facere repellendus voluptates quia ratione harum vitae ut

eligendi iste nostrum consequuntur adipisci praesentium sit beatae perferendis

similique fugit est illum et dolorum harum et voluptate eaque quidem exercitationem quos nam commodi possimus cum odio nihil nulla dorum exercitationem magnam ex et a et distinctio debitis

optio dolor molestias sit

temporibus est consectetur dolore et libero debitis vel velit laboriosam quia ipsum quibusdam qui itaque fuga rem aut ea et iure quam sed maxime ut distinctio quae

ut numquam possimus omnis eius suscipit laudantium iure

est natus reiciendis nihil possimus aut provident ex et dolor repellat pariatur est nobis rerum repellendus dolorem autem

aut quo modi neque nostrum ducimus

voluptatem quisquam iste voluptatibus natus officiis facilis dolorem quis quas ipsam vel et voluptatum in aliquid

quibusdam cumque rem aut deserunt

voluptatem assumenda ut qui ut cupiditate aut impedit veniam occaecati nemo illum voluptatem laudantium molestiae beatae rerum ea iure soluta nostrum eligendi et voluptate

ut voluptatem illum ea doloribus itaque eos

voluptates quo voluptatem facilis iure occaecati vel assumenda rerum officia et illum perspiciatis ab deleniti laudantium repellat ad ut et autem reprehenderit

laborum non sunt aut ut assumenda perspiciatis voluptas

inventore ab sint natus fugit id nulla sequi architecto nihil quaerat eos tenetur in in eum veritatis non quibusdam officiis aspernatur cumque aut commodi aut

Figure 12: Blog Posts output — screenshot 6 of 7

repellendus qui recusandae incidunt voluptates tenetur qui omnis exercitationem

error suscipit maxime adipisci consequuntur recusandae voluptas eligendi et est et voluptates quia distinctio ab amet quaerat molestiae et vitae adipisci impedit sequi nesciunt quis consectetur

soluta aliquam aperiam consequatur illo quis voluptas

sunt dolores aut doloribus dolore doloribus voluptates tempora et doloremque et quo cum asperiores sit consectetur dolorem

qui enim et consequuntur quia animi quis voluptate quibusdam

usto est quibusdam fuga quas quaerat molestias a enim ut sit accusamus enim temporibus iusto accusantium provident architecto soluta esse reprehenderit qui laborum

ut quo aut ducimus alias

ninima harum praesentium eum rerum illo dolore quasi exercitationem rerum nam porro quis neque quo consequatur minus dolor quidem veritatis sunt non explicabo similique

sit asperiores ipsam eveniet odio non quia

otam corporis dignissimos vitae dolorem ut occaecati accusamus ex velit deserunt et exercitationem vero incidunt corrupti mollitia

sit vel voluptatem et non libero

debitis excepturi ea perferendis harum libero optio eos accusamus cum fuga ut sapiente repudiandae et ut incidunt omnis molestiae nihil ut eum odit

qui et at rerum necessitatibus

ut est omnis dolores neque rerum quod ea rerum velit pariatur beatae excepturi et provident voluptas corrupti corporis harum reprehenderit dolores eligendi

Figure 13: Blog Posts output — screenshot 7 of 7

Conclusion

The Posts component successfully mounts, fetches data from the remote API in `componentDidMount()`, and renders the returned list of posts. The application runs without errors, confirming that the life cycle hooks were implemented and sequenced correctly.